



OPEN RADIO PLATFORM TECHNICAL REFERENCE

N0JCG Pluto Firmware API Contract

HTTP routes, payloads, status fields, and integration rules

RELEASE	v0.39-N0JCG.5f	PUBLICATION	August 2026
PLATFORM	PlutoSDR / Pluto Plus	AUDIENCE	Operators and app builders

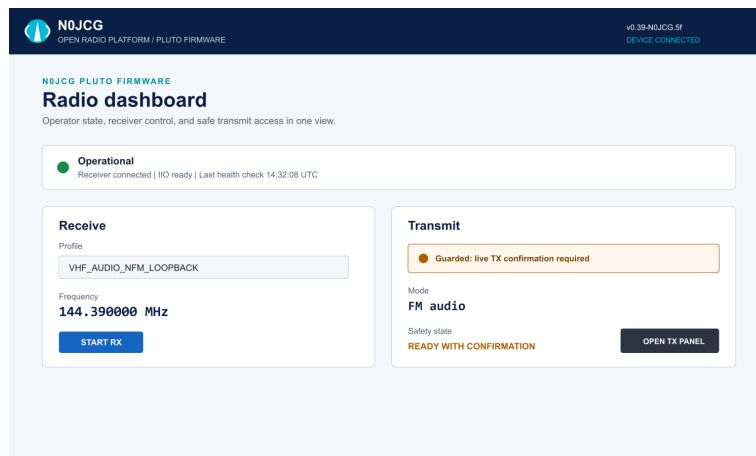
A practical N0JCG reference for firmware operation, application integration, diagnostics, and safe RF workflows.

Contents

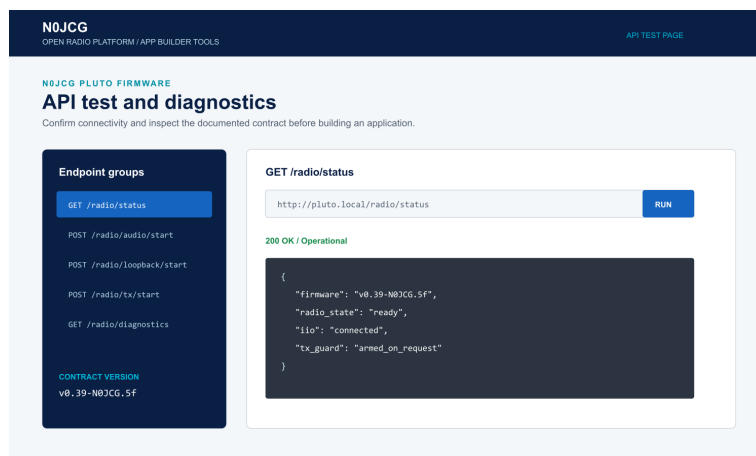
Radio API Contract
Transport
App Builder Quick Start
Endpoint Summary
Profiles
Guardrails and Calibration
Self Test
System
Radio
Decoded Audio
Capture
Spectrum
Loopback Diagnostics
Transmit
Doppler
CLI Equivalents
Onboard Test Page
Validation

Interface reference

These branded screen references show the operator dashboard and app-builder API test surface used by the Pluto firmware workflow.



Operator dashboard - system state, receive controls, and guarded transmit access.



API test page - endpoint groups, request controls, and a successful status response.

Transport

The current firmware uses a resident Python API process behind lighttpd:

- 1 `pluto-radio-api` serve `host=127.0.0.1` `port=8081` owns the firmware API.
- 1 lighttpd serves static files from `/www` on port 80.
- 1 lighttpd reverse-proxies API routes from port 80 to the resident API.
- 1 lighttpd streams long-lived proxy responses to clients as they arrive; it does not wait for an audio or spectrum stream to finish before sending it.
- 1 Browser clients and host-side tools normally use the port 80 lighttpd surface.
- 1 On-device applications should use the resident API directly with `base_url=http://127.0.0.1:8081`.

Base URLs:

```
External/browser base URL: http://192.168.2.1
On-device direct base URL: http://127.0.0.1:8081
```

Use logical API routes directly:

```
GET http://192.168.2.1/system/health
POST http://192.168.2.1/radio/profile/apply
GET http://127.0.0.1:8081/system/health
POST http://127.0.0.1:8081/radio/profile/apply
```

POST bodies should be JSON:

```
Content-Type: application/json
```

All control and status endpoints return JSON. Successful responses include `"ok": true`. Errors use the same JSON shape:

```
{
  "ok": false,
  "error": {
    "code": "out_of_range",
    "message": "frequency_hz must be between 70000000 and 600000000",
    "details": {
      "frequency_hz": 1
    }
  }
}
```

The HTTP handler accepts GET, POST, PUT, and PATCH; documented control operations use GET and POST. Query parameters are parsed for every method. For POST, PUT, and PATCH, a JSON body is merged into the query payload, with body fields taking precedence. Unsupported routes return HTTP 404 with `error=not_found` and the requested method/path in `details`.

Streaming exceptions

These paths bypass the JSON route dispatcher and return a streaming response:

Method	Path	Response
GET	<code>/radio/audio/live.pcm</code>	signed 16-bit little-endian mono PCM
GET	<code>/radio/audio/live.wav</code>	WAV containing the same PCM stream
GET	<code>/radio/spectrum/stream</code>	newline-delimited JSON (<code>application/x-ndjson</code>)

Method	Path	Response
GET	/capture/download	stored capture bytes with download headers

The short aliases `/audio/live.pcm`, `/audio/live.wav`, and `/spectrum/stream` are accepted by the current handler. Capture download has no short alias.

App Builder Quick Start

Read health:

```
curl "http://192.168.2.1/system/health"
```

List profiles:

```
curl "http://192.168.2.1/radio/profile/list"
```

Apply a profile:

```
curl -X POST "http://192.168.2.1/radio/profile/apply" \
  -H "Content-Type: application/json" \
  -d "{\"profile\":\"SAT_AUDIO_NFM\",\"frequency_hz\":145800000}"
```

Start decoded audio in simulation:

```
curl -X POST "http://192.168.2.1/radio/audio/start" \
  -H "Content-Type: application/json" \
  -d "{\"profile\":\"NOAA_NFM\",\"simulate\":true}"
```

Run a bounded TX test in simulation:

```
curl -X POST "http://192.168.2.1/radio/tx/start" \
  -H "Content-Type: application/json" \
  -d "{\"profile\":\"TX_TEST_TONE\",\"simulate\":true}"
```

Check guardrails before TX:

```
curl -X POST "http://192.168.2.1/radio/tx/guardrails" \
  -H "Content-Type: application/json" \
  -d "{\"profile\":\"TX_AUDIO_FM\",\"simulate\":true}"
```

Run the safe self-test bundle:

```
curl -X POST "http://192.168.2.1/system/self-test" \
  -H "Content-Type: application/json" \
  -d "{}"
```

Run a bounded live TX test only in a controlled RF setup:

```
curl -X POST "http://192.168.2.1/radio/tx/start" \
  -H "Content-Type: application/json" \
  -d "{\"profile\":\"TX_TEST_TONE\",\"confirm_live_tx\":true}"
```

Live TX emits RF. Use a dummy load, shielded setup, or a legal test frequency.

Sample app clients are available in `examples/python/pluto_radio_client.py` and `examples/browser/pluto-radio-client.html`.

Endpoint Summary

System:

```
GET /system/health
GET /system/metrics
GET /system/settings
POST /system/settings
GET /system/files
GET /system/logs
GET /system/watchdog
POST /system/watchdog/check
POST /system/self-test
GET /system/self-test/status
```

Radio and profiles:

```
GET /radio/status
GET /radio/profile/list
POST /radio/profile/apply
POST /radio/tune
POST /radio/stop
```

Decoded audio:

```
GET /radio/audio/status
POST /radio/audio/start
POST /radio/audio/retune
POST /radio/audio/demod-self-test
POST /radio/audio/stop
GET /radio/audio/live.pcm
GET /radio/audio/live.wav
```

Capture:

```
POST /capture/start
GET /capture/list
GET /capture/metadata
GET /capture/download
POST /capture/delete
```

Spectrum:

```
GET /radio/spectrum/status
GET /radio/spectrum/snapshot
GET /radio/spectrum/top
GET /radio/spectrum/stream
POST /radio/spectrum/snapshot
POST /radio/spectrum/top
```

Loopback diagnostics:

```
GET /radio/loopback/status
GET /radio/loopback/demod/status
GET /radio/loopback/cw/status
GET /radio/loopback/ft8/status
POST /radio/loopback/start
POST /radio/loopback/demod
POST /radio/loopback/cw
POST /radio/loopback/ft8
```

Transmit:

```
GET /radio/tx/status
GET /radio/tx/guardrails
POST /radio/tx/guardrails
POST /radio/tx/start
POST /radio/tx/stop
```

Calibration:

```
GET /radio/calibration/status
POST /radio/calibration/apply
POST /radio/calibration/measure
```

Doppler:

```
GET /radio/doppler/status
POST /radio/doppler/plan
POST /radio/doppler/start
POST /radio/doppler/stop
POST /radio/doppler/tick
```

Current short aliases are retained for existing clients:

```
/status, /profiles, /health
/dashboard/metrics, /dashboard/settings, /dashboard/files
/logs, /watchdog, /watchdog/check
/captures, /capture/meta
/spectrum/status, /spectrum/snapshot, /spectrum/top, /spectrum/stream
/loopback/status, /loopback/start, /loopback/demod, /loopback/cw, /loopback/ft8
/tx/status, /tx/start, /tx/stop, /tx/guardrails
/calibration/status, /calibration/apply, /calibration/measure
/self-test/status, /self-test
/doppler/status, /doppler/plan, /doppler/start, /doppler/stop, /doppler/tick
/apply, /tune, /stop
/audio/status, /audio/start, /audio/retune, /audio/demod-self-test,
/audio/stop, /audio/live.pcm, /audio/live.wav
```

The firmware also accepts `POST /capture` as an alias for `POST /capture/start`. It does not accept `POST /capture/download`.

Profiles

Profiles are JSON files loaded from:

```
/mnt/jffs2/pluto-radio/profiles
/etc/pluto-radio/profiles
```

The persistent `/mnt/jffs2` directory wins, so applications can install or override profiles without replacing the firmware image.

Common fields:

```
{
  "name": "SAT_AUDIO_NFM",
  "label": "Satellite audio narrowband FM",
  "default_frequency_hz": 145800000,
  "sample_rate_hz": 520999,
  "rf_bandwidth_hz": 521000,
  "fir_enabled": true,
  "gain_control_mode": "manual",
  "gain_db": 55,
  "demod_mode": "nfm",
  "audio_rate_hz": 48000,
  "filter_width_hz": 16100,
  "frequency_shift_hz": 400,
  "stream_format": "pcm_s16le",
  "tx_allowed": false
}
```

Profiles for externally translated RF may include:

```
{
  "name": "CB_AM_HAMITUP",
  "source_frequency_hz": 27185000,
  "frequency_translation_hz": 125000000,
  "translated_frequency_hz": 152185000,
  "default_frequency_hz": 152185000
}
```

`default_frequency_hz` is always the Pluto LO frequency and must remain within the Pluto tuning range. For upconverter profiles, `translated_frequency_hz` must equal `source_frequency_hz + frequency_translation_hz`. Apps may display `source_frequency_hz` to the user while sending the translated/default Pluto frequency to firmware.

TX-capable profiles add explicit transmit fields:

```
{
  "name": "TX_TEST_TONE",
  "tx_allowed": true,
  "tx_mode": "fm",
  "tx_duration_limit_seconds": 10,
  "tx_gain_db": -30.0,
  "tx_tone_hz": 0,
  "tx_amplitude": 0.05,
  "tx_audio_source": "tone",
  "tx_audio_rate_hz": 8000,
  "tx_audio_tone_hz": 1000,
  "tx_fm_deviation_hz": 5000
}
```

Audio and CW TX profiles add mode-specific fields:

```
{
  "name": "TX_AUDIO_FM",
  "tx_mode": "fm",
  "tx_audio_source": "tone",
  "tx_audio_rate_hz": 8000,
  "tx_audio_tone_hz": 1000,
  "tx_fm_deviation_hz": 5000
}
```

FT8 loopback TX profiles add the encoded message and base audio frequency:


```
{
  "name": "TX_FT8_LOOPBACK",
  "tx_mode": "ft8",
  "tx_ft8_text": "CQ K1ABC FN42",
  "tx_audio_tone_hz": 1500,
  "tx_duration_limit_seconds": 30
}
```

Applications can provide audio by writing signed 16-bit little-endian mono PCM to a file or FIFO under `/mnt/jffs2`, `/media`, `/tmp`, or `/var/run`, then starting TX with `tx_audio_source=file` and `tx_audio_path=/path/to/audio.pcm`. For AM/FM tone-source requests, `tx_tone_hz` is accepted as a compatibility alias for `tx_audio_tone_hz` when `tx_audio_tone_hz` is omitted.

Validation bounds:

```
frequency_hz: 70 MHz to 6 GHz
sample_rate_hz: 520.833 kSPS to 61.44 MSPS, validated against the active
AD9361 clock/filter path
rf_bandwidth_hz: 200 kHz to 56 MHz
tx_gain_db: -89.75 dB to 0 dB
tx_amplitude: 0.0 to PLUTO_TX_MAX_AMPLITUDE, default 0.25
tx_audio_rate_hz: 8 kHz to 48 kHz
tx_audio_tone_hz: 20 Hz to 3 kHz
tx_am_modulation_index: 0.0 to 1.0
tx_fm_deviation_hz: 100 Hz to 25 kHz and below sample_rate_hz / 8
tx_cw_wpm: 5 to 40
tx_ft8_text: 1 to 35 printable ASCII characters
tx_cw_text: 1 to 80 characters from the firmware CW alphabet
tx_tone_hz: 0 to 200 kHz; tone mode additionally requires at least 100 Hz and
must remain below sample_rate_hz / 4
duration_seconds: 1 to the active profile and PLUTO_TX_MAX_SECONDS limits
loopback_duration_limit_seconds: 0 to PLUTO_LOOPBACK_MAX_SECONDS, default 10
```

The AD9361 driver computes the valid sample-rate range from the active clock and FIR filter path. Profiles may request `fir_enabled=true`; firmware applies `in_out_voltage_filter_fir_en=1` before validating and writing `in_voltage_sampling_frequency`, so low-rate profiles use the same FIR-decimated path reported by `in_voltage_sampling_frequency_available`.

Bundled NOAA_NFM defaults are aligned with the validated SDRSharp receive path for NOAA weather testing: `fir_enabled=true`, `sample_rate_hz=520999`, `rf_bandwidth_hz=521000`, `gain_db=55`, `filter_width_hz=16100`, `frequency_shift_hz=400`, `deemphasis=75us`, and `output_gain=2.0`. The `frequency_shift_hz` value is a DSP demodulation offset; it does not change `requested_frequency_hz`. The production NFM backend also uses FM channel filtering and limiting before quadrature demodulation; apps do not need to implement these receiver-DSP details.

Bundled audio receive profiles use the same validated low-rate FIR path unless the modulation requires a wider bandwidth. `SAT_AUDIO_NFM`, `VHF_AUDIO_NFM_LOOPBACK`, and `UHF_AUDIO_NFM_LOOPBACK` are narrowband FM audio profiles. `SAT_CW`, `VHF_CW_LOOPBACK`, and `UHF_CW_LOOPBACK` are CW monitor profiles with a browser-friendly BFO tone and live `audio.cw_decode` status. `CB_AM_HAMITUP` is an AM audio receive profile for CB channel 19 through a Ham It Up +125 MHz upconverter; it is a translated live-RX profile, not a Pluto TX/RX loopback profile. `FT8_40M_HAMITUP` uses the same converter convention for the 7.074 MHz 40 m FT8 dial frequency, tunes Pluto to 132.074 MHz, and selects the native USB/FT8 decoder at 12 kHz audio rate.

Guardrails and Calibration

GET `/radio/tx/guardrails`

Returns the current TX limits, supported modes, safe audio roots, and the live TX confirmation rule. Apps should use this endpoint to populate UI bounds.

POST `/radio/tx/guardrails`

Request:

```
{
  "profile": "TX_AUDIO_FM",
  "simulate": true,
  "duration_seconds": 1
}
```

Returns a validated plan plus readiness fields:

```
{
  "ok": true,
  "plan": {},
  "readiness": {
    "simulation_ready": true,
    "live_ready": false,
    "radio_present": true,
    "confirmation_required": false,
    "blockers": [],
    "warnings": []
  }
}
```

GET /radio/calibration/status

Returns persisted calibration metadata and the same guardrails. RX calibration is firmware-managed: clients request the desired channel center in `frequency_hz`, and firmware applies `rx_frequency_offset_hz` when programming the AD9361 RX LO. Apps should not scan or shop around nearby frequency shifts for ordinary NOAA or satellite use.

POST /radio/calibration/apply

Request:

```
{
  "rx_frequency_offset_hz": 12.5,
  "tx_frequency_offset_hz": 0,
  "rx_gain_offset_db": 0,
  "tx_gain_offset_db": -1,
  "notes": "bench note"
}
```

Bounds:

```
rx_frequency_offset_hz, tx_frequency_offset_hz: -250 kHz to +250 kHz
rx_gain_offset_db, tx_gain_offset_db: -20 dB to +20 dB
notes: 200 characters
```

Behavior:

- 1 `rx_frequency_offset_hz` is persisted and automatically applied to all RX tune paths: `/radio/profile/apply`, `/radio/tune`, `/radio/audio/retune`, audio start requests that include tuning fields, and Doppler retune ticks.
- 1 If a channel is already tuned when calibration is changed, firmware retunes the current `requested_frequency_hz` immediately using the new offset.
- 1 `radio.requested_frequency_hz` remains the user/app requested channel center. `radio.hardware_frequency_hz` and `radio.actual_frequency_hz` report the LO value written/read from the AD9361 after calibration is applied.
- 1 `frequency_shift_hz` remains a DSP/off-center-demod control. It is not the normal way to compensate for device LO calibration.

POST /radio/calibration/measure

Measures RX frequency calibration from a known external reference carrier and returns a recommended persistent `rx_frequency_offset_hz`. This is the preferred calibration workflow; clients should not guess offsets by shopping around channels.

Request:

```
{
  "profile": "NOAA_NFM",
  "reference_frequency_hz": 162500000,
  "gain_db": 45,
  "gain_control_mode": "manual",
  "span_hz": 50000,
  "search_hz": 25000,
  "bins": 1024,
  "measurements": 3,
  "min_snr_db": 8,
  "apply": false
}
```

Behavior:

- 1 Stops any running audio session before measuring, because RX streaming and spectrum capture cannot safely share the AD9361 receive path.
- 1 Tunes the selected profile to `reference_frequency_hz`, using the currently persisted `rx_frequency_offset_hz`.
- 1 Captures one or more spectrum snapshots, finds the strongest in-window peak, and computes:
 - 1 `measured_peak_offset_hz = peak_frequency_hz - reference_frequency_hz`
 - 1 `recommended_rx_frequency_offset_hz = current_rx_frequency_offset_hz + measured_peak_offset_hz`
- 1 If `apply=true`, persists the recommended offset by calling the same behavior as `/radio/calibration/apply` and retunes the current requested frequency.
- 1 Requires an external known-good RF reference. A Pluto TX-to-RX loopback is useful for demod/audio testing, but it is not an absolute frequency calibration reference because TX and RX share the same clock.

Response:

```
{
  "ok": true,
  "calibration_measurement": {
    "profile": "NOAA_NFM",
    "reference_frequency_hz": 162500000,
    "span_hz": 50000,
    "search_hz": 25000,
    "bins": 1024,
    "bin_width_hz": 48.88,
    "measurements_requested": 3,
    "measurements_used": 3,
    "current_rx_frequency_offset_hz": 0,
    "measured_peak_offset_hz": -850,
    "recommended_rx_frequency_offset_hz": -850,
    "confidence": "high",
    "samples": [
      {
        "measurement": 1,
        "frequency_hz": 162499150,
        "measured_peak_offset_hz": -850,
        "power_dbfs": -42.1,
        "snr_db": 31.5
      }
    ],
    "rejected": [],
    "applied": null
  }
}
```

Self Test

POST /system/self-test

Runs a safe diagnostic bundle. It checks profiles, health surfaces, guardrails, calibration status, simulated audio, simulated spectrum, simulated loopback, and simulated tone/AM/FM/CW TX. It does not run live RF.

Optional request:

```
{
  "require_radio": false,
  "tx_duration_seconds": 1
}
```

Optional live bench request for a Pluto with a loopback cable and suitable attenuation installed:

```
{
  "require_radio": true,
  "live_loopback": true,
  "live_spectrum": true,
  "confirm_live_tx": true,
  "loopback_frequency_hz": 915000000,
  "cw_wpm_values": [12, 20],
  "cw_text": "CQ TEST",
  "spectrum_center_frequency_hz": 915000000,
  "spectrum_span_hz": 200000,
  "spectrum_bins": 64,
  "spectrum_frames": 5,
  "spectrum_interval_ms": 100
}
```

`live_loopback=true` runs bounded CW RF loopback decoder checks and requires `confirm_live_tx=true`.

`cw_wpm_values` accepts 1 to 6 speeds from 5 to 40 WPM; the default bench sweep is `[12, 20]`. If

`cw_duration_seconds` is omitted, firmware estimates a bounded duration from `cw_text` and WPM so slow-speed tests capture a complete message plus setup margin. If `cw_duration_seconds` is provided, firmware uses the caller's explicit bounded duration. `cw_capture_seconds` defaults to the full chosen duration. The live CW decoder retains up to the firmware bounded TX limit, so low-speed self-tests are not truncated by the decoder window. `live_spectrum=true` runs a bounded live spectrum stream smoke test and reports row count, sequence range, center frequency, bin count, backend name, and per-row `frame_duration_ms` values. Live spectrum does not transmit, but it does retune/configure the receiver.

The response includes `self_test.state` as `pass`, `warn`, or `fail` and a per-step result list for app diagnostics.

System

GET /system/health

Returns a single JSON view for dashboards and app readiness checks:

```
{
  "ok": true,
  "system": {
    "hostname": "pluto",
    "uptime_seconds": 1234,
    "api_mode": "lighttpd-python-http",
    "api_version": "0.1"
  },
  "radio": {},
  "audio": {},
  "watchdog": {},
  "loopback": {},
  "tx": {},
  "tx_guardrails": {},
  "tx_readiness": {},
  "calibration": {},
  "self_test": {},
  "doppler": {},
  "iio": {}
}
```

GET /system/logs

Query parameters:

```
source=messages|syslog|audio|profile_check|dmesg
lines=1..500
```

The response is bounded JSON and should be safe for app log panels.

Radio

GET /radio/status

Returns the active profile, requested/actual frequency, RX/TX IIO attributes, stream state, and last error.

POST /radio/profile/apply

Request:

```
{
  "profile": "SAT_AUDIO_NFM",
  "frequency_hz": 145800000,
  "gain_db": 40,
  "gain_control_mode": "manual"
}
```

Behavior:

- 1 Applies RX LO, RX bandwidth, RX sample rate, gain mode, and manual gain.
- 1 Stores state in /var/run/pluto-radio/state.json.
- 1 Rejects invalid profiles and out-of-range values.

POST /radio/tune

Request:

```
{
  "frequency_hz": 145800000
}
```

Updates only the RX LO and preserves the current profile.

POST /radio/stop

Stops audio, forces TX idle when possible, and moves radio state to idle.

Decoded Audio

POST /radio/audio/start

Request:

```
{
  "profile": "NOAA_NFM",
  "frequency_hz": 162550000,
  "simulate": true,
  "squelch_db": -58,
  "deemphasis": "75us",
  "agc": "fast_attack",
  "frequency_shift_hz": 400,
  "iq_mode": "normal",
  "cw_decode_wpm": 12,
  "output_gain": 2.0,
  "noise_gate_db": -85,
  "dc_block": false
}
```

Behavior:

- 1 Validates the selected profile has an audio demod mode.
- 1 For live production audio, applies the selected profile/frequency and forces the RX chain into `ensm_mode=fdd` before streaming.
- 1 `frequency_shift_hz` is optional and defaults to 0. It asks the firmware DSP to mix the received complex I/Q before demodulation, which lets an app tune the RX LO off-center while keeping demodulation centered in firmware. Valid range is plus/minus half of the selected profile sample rate. Apps should normally use the profile default; for NOAA_NFM that default is 400 Hz based on the validated local receive path.
- 1 `iq_mode` is optional and defaults to `normal`. It is a firmware diagnostic control for live I/Q interpretation. Valid values are `normal`, `swap`, `invert_i`, `invert_q`, `conjugate`, `invert_both`, `swap_invert_i`, `swap_invert_q`, and `swap_invert_both`. Applications should leave this at `normal` unless firmware diagnostics identify an I/Q ordering or sign issue.
- 1 For CW profiles such as SAT_CW, firmware enables a live rolling CW decoder inside the production audio backend. `cw_decode_wpm` is an optional timing hint from 0 to 60 WPM; 0 or omission means firmware starts from an adaptive default and estimates timing from the received keying.
- 1 Starts `/usr/sbin/pluto-audio-backend` for production audio.
- 1 Uses `/usr/sbin/pluto-audio-sim-backend` when `simulate=true`.
- 1 If audio is already running, an identical start request is idempotent and returns the current state.
- 1 If audio is already running with the same profile, demod, controls, and backend mode, `frequency_hz`, `gain_db`, and `gain_control_mode` are applied as an in-place retune without restarting the DSP backend.
- 1 A start request that changes profile, demod controls, `frequency_shift_hz`, or backend mode stops the current backend and launches the requested backend. Applications should not manually sequence stop/apply/start for ordinary retunes.
- 1 `audio.backend` is a status enum:
 - 1 `external`: production or environment-overridden backend.
 - 1 `simulated_pcm`: `/usr/sbin/pluto-audio-sim-backend` selected by `simulate=true`.
 - 1 `dry_run`: host validation mode.
- 1 Tracks state in `/var/run/pluto-radio/audio.json`.

- 1 Production audio backend startup failures are reported through `audio.last_error` and force `audio.state` to `error`. The backend retries an interrupted FIFO sink open (EINTR) instead of silently becoming a live but non-producing process. Fatal backend startup details are also mirrored through `/var/run/pluto-radio/audio-backend-status.json` for diagnostics. While running, the same sidecar reports backend progress fields such as `audio.iio_refills`, `audio.pcm_bytes`, `audio.pcm_rate_hz`, `audio.pcm_measured_rate_hz`, `audio.input_sample_rate_hz`, `audio.frequency_shift_hz`, `audio.fm_channel_filter`, `audio.fm_limiter`, `audio.processing_sample_rate_hz`, `audio.iq_decimation`, `audio.phase`, `audio.rms_level`, `audio.cw_decode`, `audio.ft8_decode`, and `audio.squelch_state`; a live session that cannot refill an IIO buffer is converted to `audio.state=error` instead of remaining silently idle.

Audio Level And Squelch Status

GET `/radio/audio/status` and the audio object embedded in GET `/system/health` expose live audio telemetry from the production backend. The two fields most applications use for activity indicators are:

- 1 `audio.rms_level`: null before the backend has produced its first status report, after audio is stopped, or during startup/error paths that have no PCM measurement yet. While streaming, it is a JSON number representing the recent decoded PCM RMS level as a linear full-scale ratio. It is not dBFS. Typical values are small decimals such as 0, 0.00058, 0.01658822, or 0.10376281. If an app wants dBFS, compute $20 * \log_{10}(\max(\text{audio.rms_level}, \text{epsilon}))$.
- 1 `audio.squelch_state`: a string enum. Valid values are:
 - 1 `unknown`: no backend report has been received yet, audio is stopped, or an error/startup path could not determine squelch state.
 - 1 `disabled`: squelch is intentionally disabled. Current firmware treats `squelch_db <= -119` as disabled; common test payloads use -120.
 - 1 `open`: squelch is enabled and the backend is currently passing audio. The backend includes a short hang interval, so open can persist briefly after the level falls below threshold.
 - 1 `closed`: squelch is enabled and the backend is currently muting audio because signal level is below `squelch_db`.

`audio.squelch_state` describes the squelch gate only. `noise_gate_db` is a separate post-demod audio gate and does not change the squelch enum.

Example stopped or pre-first-report status:

```
{
  "audio": {
    "state": "stopped",
    "rms_level": null,
    "squelch_state": "unknown"
  }
}
```

Example live status with squelch disabled:

```
{
  "audio": {
    "state": "running",
    "profile": "NOAA_NFM",
    "rms_level": 0.01658822,
    "squelch_state": "disabled",
    "pcm_rate_hz": 48000,
    "pcm_measured_rate_hz": 47980.5
  }
}
```

Example live status with squelch enabled:

```
{
  "audio": {
    "state": "running",
    "profile": "SAT_CW",
    "rms_level": 0.00074578,
    "squelch_state": "open"
  }
}
```

- 1 When a CW profile is running, `audio.cw_decode` is either `null` or an object with live rolling decoder state:

```
{
  "decode_supported": true,
  "requested_wpm": 12,
  "estimated_wpm": 12,
  "estimated_unit_ms": 100,
  "keyed_percent": 41.2,
  "keying_segments": 29,
  "envelope_floor": 0.000001,
  "envelope_peak": 0.02,
  "envelope_threshold": 0.009,
  "current_symbol": "...",
  "decoded_symbols": "-.-. --- / - . . . - ",
  "decoded_text": "CQ TEST",
  "timing_source": "streaming_auto"
}
```

`decoded_text` and `decoded_symbols` are rolling in-memory fields from the current live audio session; they reset when the audio backend restarts. Apps should treat `current_symbol` as provisional and display `decoded_text` as the confirmed text stream.

- 1 When FT8_40M_HAMITUP is running, `audio.ft8_decode` exposes the most recent completed 15-second receive slot. The first partial slot after backend startup is discarded. Candidate decoding runs on a worker thread so IIO refills continue while the previous slot is decoded:

```
{
  "decode_supported": true,
  "protocol": "FT8",
  "state": "collecting",
  "slot_utc": "2026-07-19T00:15:00Z",
  "slots_completed": 4,
  "slots_dropped": 0,
  "decode_time_ms": 327.4,
  "candidate_count": 12,
  "message_count": 2,
  "messages": [
    {
      "time_offset_s": 0.32,
      "audio_frequency_hz": 1043.8,
      "sync_score": 18,
      "text": "CQ K1ABC FN42"
    }
  ]
}
```

`sync_score` is the decoder's Costas synchronization score; it is not an SNR estimate. `slots_dropped` increments if a decode overruns the next slot.

- 1 The production backend reads Pluto AD9361 RX buffers using the kernel-reported `le:S12/16>>0` scan element format. Firmware sign-extends those 12-bit I/Q samples before demodulation; applications receive decoded PCM and do not need to handle raw AD9361 sample packing.
- 1 If the external backend reports active IIO streaming while AD9361 `ensm_mode` is not `fdd` or `rx`, the API reports `audio.state=error` with `audio.last_error.code=rx_chain_not_streaming`. This prevents dashboards from treating a wedged RF chain as healthy only because the backend process is alive.

POST /radio/audio/retune

Request:

```
{
  "profile": "NOAA_NFM",
  "frequency_hz": 162550000,
  "gain_db": 42,
  "gain_control_mode": "manual"
}
```

Behavior:

- 1 Intended for live frequency/gain changes while audio is running.
- 1 Only updates RX LO and optional gain controls; it does not change sample rate, RF bandwidth, demod mode, filter width, squelch, backend, or audio format.
- 1 Forces ensm_mode=fdd after the retune and preserves stream_state=audio when a session is active.
- 1 Returns 409 audio_retune_profile_mismatch if asked to retune a different profile than the running audio session. Use /radio/audio/start for profile or demod changes.

POST /radio/audio/demod-self-test

Runs a bounded firmware DSP self-test without transmitting RF. The API starts /usr/sbin/pluto-audio-backend with PLUTO_AUDIO_SOURCE=synthetic_fm, feeds it synthetic FM-modulated I/Q, reads decoded PCM from a temporary FIFO, and reports the recovered audio tone metrics. Use this to separate demodulator faults from antenna, cable, TX leakage, and AD9361 direct-conversion behavior.

Request:

```
{
  "profile": "NOAA_NFM",
  "duration_seconds": 3,
  "capture_seconds": 3,
  "tone_hz": 1000,
  "fm_deviation_hz": 5000,
  "carrier_hz": 0,
  "frequency_shift_hz": 0,
  "simulate": true
}
```

Response:

```
{
  "ok": true,
  "bounded": true,
  "demod_self_test": {
    "profile": "NOAA_NFM",
    "source": "synthetic_fm",
    "tone_hz": 1000,
    "fm_deviation_hz": 5000,
    "carrier_hz": 0,
    "frequency_shift_hz": 0,
    "metrics": {
      "sample_count": 144000,
      "duration_seconds": 3.0,
      "pcm_bytes": 288000,
      "rms_dbfs": -18.0,
      "peak_dbfs": -6.0,
      "tone_hz": 1000,
      "detected_tone_hz": 1000,
      "tone_error_hz": 0,
      "tone_dbfs": -14.0,
      "reference_dbfs": -58.0,
      "tone_snr_db": 44.0,
      "passed": true
    }
  }
}
```

`carrier_hz` may be paired with `frequency_shift_hz` to verify the DSP frequency-shift path used by cabled loopback demod diagnostics.

The self-test is bounded. If the backend does not produce the requested PCM before the deadline, firmware returns `503 demod_self_test_timeout` with diagnostic details including bytes produced, target bytes, backend status, and a backend log tail. It must not leave a background backend running.

Audio streams:

```
GET /radio/audio/live.pcm
GET /radio/audio/live.wav
```

Applications should prefer `live.wav` for browser playback and `live.pcm` for native clients that already know the sample format.

Decoded audio is mono signed 16-bit little-endian PCM. `audio_rate_hz` from `/radio/audio/status` is the exact PCM sample rate; the `live.wav` header uses the same value. This is the PCM format rate written into the stream, not a hard real-time minimum delivery guarantee over HTTP, USB Ethernet, or the browser media pipeline.

`pcm_rate_hz` is the configured decoded PCM sample rate and should match `audio_rate_hz`, for example 48000 for NOAA_NFM. `pcm_measured_rate_hz` is diagnostic reader-throughput telemetry from the latest backend report interval; it can drop when no client is draining the FIFO and must not be used as the WAV format rate. `input_sample_rate_hz`, `processing_sample_rate_hz`, and `iq_decimation` expose the raw AD9361 rate and the firmware's CPU-saving I/Q pre-decimation; applications should treat them as diagnostics and continue using `audio_rate_hz/pcm_rate_hz` as the PCM/WAV format rate.

Browser clients should treat continuous audio as a real-time stream with normal embedded-device jitter. For smooth playback, start `/radio/audio/live.wav?continuous=true`, build an initial playback buffer, and use application-side de-jitter/reconnect handling. The firmware exposes `pcm_measured_rate_hz` so clients can warn or recover if sustained delivery falls materially below the declared PCM rate, but the contract does not promise that every wall-clock interval will deliver at least 48,000 samples per second.

`live.wav` returns a normal HTTP response with `Content-Type: audio/wav`, `Cache-Control: no-store`, a blank line, and then the WAV RIFF body. Clients should not need to special-case a status-line-only WAV response.

When seconds is omitted, current firmware returns a bounded default clip. In the tested build this default is approximately 2 seconds. Browser audio elements that need long-lived playback must explicitly request continuous streaming with `/radio/audio/live.wav?continuous=true` or `/radio/audio/live.wav?seconds=0`.

Clients may request `seconds=1..3600` for bounded snapshot captures. Bounded responses end after the requested capture window. They may omit Content-Length on the port-80 lighttpd surface, so clients must not treat a missing Content-Length as proof that the stream is continuous.

Continuous responses intentionally omit a fixed Content-Length; on the external port-80 surface they may use HTTP/1.1 chunked transfer encoding. Headers and the WAV RIFF header are sent immediately. For continuous WAV, the firmware uses a large WAV data length in the RIFF header and keeps writing PCM until the client disconnects or the backend exits. A client timeout is therefore an expected way to end a probe of a live stream, not an API error.

POST /radio/audio/stop

Stops the backend and removes the live FIFO.

Capture

POST /capture/start

Request:

```
{
  "profile": "IQ_CAPTURE",
  "type": "iq",
  "duration_seconds": 10,
  "max_bytes": 1048576,
  "simulate": true
}
```

Behavior:

- 1 Stores capture metadata next to capture data.
- 1 Allows storage only under `/mnt/jffs2` or `/media`.
- 1 Enforces `PLUTO_CAPTURE_MAX_SECONDS` and `PLUTO_CAPTURE_MAX_BYTES`.
- 1 Uses `/usr/sbin/pluto-capture-backend` when installed.

Use SD-card storage under `/media` for larger captures.

Spectrum

GET /radio/spectrum/snapshot POST /radio/spectrum/snapshot

Request:

```
{
  "profile": "IQ_CAPTURE",
  "center_frequency_hz": 162550000,
  "span_hz": 200000,
  "bins": 256,
  "top_n": 5,
  "simulate": true
}
```

Returns bounded spectrum points and peak information.

Spectrum power field contract:

- 1 Spectrum bin power is always reported as `points[].power_dbfs`.
- 1 Peak power is always reported as `peaks[].power_dbfs`.

- 1 Peak signal-to-noise ratio is reported as `peaks[] . snr_db`.
- 1 The estimated floor for a snapshot or stream row is reported as `noise_floor_dbfs`.
- 1 These values are numeric dBFS values. More-positive values are stronger signals; for example, `-42.5 dBFS` is stronger than `-90.25 dBFS`.
- 1 Apps should treat `power_dbfs` as the canonical signal-power field for spectrum data. Aliases such as `power`, `db`, `level`, `rsi`, or `signal_power` are not part of the firmware spectrum contract.
- 1 Hardware receiver RSSI is a different measurement and appears in radio/IIO status as `rx_rssi_ch0` and `rx_rssi_ch1`, not in spectrum point arrays.

GET `/radio/spectrum/top` POST `/radio/spectrum/top`

Uses the same input but returns only the peak list. Apps should use this when they do not need the full point array.

GET `/radio/spectrum/stream`

Query parameters:

```
profile=IQ_CAPTURE
center_frequency_hz=162550000
span_hz=200000
bins=192
top_n=5
frames=120
interval_ms=250
continuous=false
follow_doppler=false
simulate=true
```

Returns `application/x-ndjson`, one JSON spectrum row per line. Each row is bounded to the requested bin count and has this shape:

```
{
  "ok": true,
  "type": "spectrum_row",
  "sequence": 0,
  "time_epoch": 1783713600,
  "sample_count": 4096,
  "sample_rate_hz": 2400000,
  "center_frequency_hz": 162550000,
  "span_hz": 200000,
  "bins": 192,
  "backend": "external_stream",
  "bounded": false,
  "continuous": true,
  "follow_doppler": false,
  "frame_duration_ms": 10,
  "points": [
    {"frequency_hz": 162450000, "power_dbfs": -90.25}
  ],
  "peaks": [
    {"frequency_hz": 162550000, "power_dbfs": -42.5, "snr_db": 47.0}
  ],
  "noise_floor_dbfs": -89.5
}
```

For normal bounded diagnostics, `frames` limits the number of rows and is capped by `PLUTO_SPECTRUM_MAX_STREAM_FRAMES`. For a live spectrum/waterfall display, set `continuous=true`; the firmware passes `frames=0` to the backend and streams until the client disconnects. The API terminates the backend when the client disconnects so reconnects do not leave an orphaned IIO buffer holder.

Before starting a hardware spectrum snapshot or stream, firmware applies the requested profile, center frequency, gain, RF bandwidth, sample rate, and FDD mode. With `follow_doppler=true`, a Doppler plan must already be loaded with POST `/radio/doppler/plan`; each stream row uses the interpolated plan frequency as `center_frequency_hz`, and the

backend retunes the RX LO before capturing that row. `frame_duration_ms` is measured backend capture/compute time for the row and is intended for UI performance diagnostics.

Loopback Diagnostics

POST `/radio/loopback/start`

Request:

```
{
  "profile": "LOOPBACK_TEST",
  "duration_seconds": 5,
  "frequency_hz": 915000000,
  "tx_gain_db": -30,
  "tx_tone_hz": 10000,
  "tx_amplitude": 0.05,
  "simulate": true
}
```

Live loopback requires:

```
{
  "confirm_live_tx": true
}
```

Behavior:

- 1 Requires `tx_allowed=true` on the profile.
- 1 Applies RX and TX configuration.
- 1 Runs a bounded TX tone and RX measurement.
- 1 Reports metrics such as sample count, RX RMS, RX peak, tone, duration, and sample rate.

Use loopback for firmware and application diagnostics. Use `/radio/tx/start` for a transmit-only test.

POST `/radio/loopback/demod`

Runs a bounded closed-loop FM or AM demod diagnostic. This is different from `/radio/loopback/start`: it uses the single-process `pluto-loopback-backend` full-duplex path so one process owns both TX and RX IIO buffers. This avoids the fragile two-process pattern where a TX backend and RX audio backend compete for AD9361/IIO streaming state. The backend transmits a bounded FM or AM audio tone, captures RX IQ through the same process, applies the requested RX IF offset, and reports whether the expected audio tone was recovered.

Recommended live cabled RF pairings:

```
{
  "rx_profile": "VHF_AUDIO_NFM_LOOPBACK",
  "tx_profile": "TX_AUDIO_FM",
  "frequency_hz": 145950000,
  "rx_gain_db": 35,
  "tx_gain_db": -25,
  "tx_amplitude": 0.20,
  "confirm_live_tx": true
}
```

```
{
  "rx_profile": "UHF_AUDIO_NFM_LOOPBACK",
  "tx_profile": "TX_AUDIO_FM",
  "frequency_hz": 915000000,
  "rx_if_offset_hz": 5000,
  "rx_gain_db": 45,
  "tx_gain_db": -30,
  "tx_amplitude": 0.15,
  "confirm_live_tx": true
}
```

For CW decoder cabled RF checks, use VHF_CW_LOOPBACK or UHF_CW_LOOPBACK with TX_CW:

```
{
  "rx_profile": "VHF_CW_LOOPBACK",
  "tx_profile": "TX_CW",
  "frequency_hz": 145800000,
  "rx_if_offset_hz": 25000,
  "tx_cw_text": "CQ TEST",
  "tx_cw_wpm": 12,
  "confirm_live_tx": true
}
```

For CB AM loopback with the current Pluto-only cabled setup, use CB_AM_HAMITUP with TX_AUDIO_AM at the translated Pluto RF frequency. This validates the firmware AM demodulator and the CB receive profile at 152.185 MHz, which corresponds to CB channel 19 at 27.185 MHz through a +125 MHz Ham It Up upconverter. It does not validate the external Ham It Up receive path unless that external converter is physically wired into the loopback path.

Browser applications consuming the CB AM loopback/simulation audio should expect a steady, clean AM-demodulated test tone, normally 1000 Hz, rather than speech. A passing browser-level result is intentionally simple: the tone plays continuously, `audio.pcm_bytes` keeps increasing while streaming, `audio.rms_level` is non-null, and `audio.squelch_state` is normally disabled when squelch is disabled for the test or open when squelch is enabled and passing audio. Smooth static, periodic silence, or repeated buffering is not the expected result for this diagnostic; it means the app is not receiving the intended demodulated test stream or the AM demod/loopback path is not locked onto the generated carrier/modulation.

```
{
  "rx_profile": "CB_AM_HAMITUP",
  "tx_profile": "TX_AUDIO_AM",
  "frequency_hz": 152185000,
  "rx_if_offset_hz": 0,
  "rx_gain_db": 55,
  "tx_gain_db": -35,
  "tx_amplitude": 0.05,
  "tx_audio_tone_hz": 1000,
  "tx_am_modulation_index": 0.8,
  "confirm_live_tx": true
}
```

General request shape:

```
{
  "rx_profile": "UHF_AUDIO_NFM_LOOPBACK",
  "tx_profile": "TX_AUDIO_FM",
  "frequency_hz": 915000000,
  "rx_if_offset_hz": 5000,
  "duration_seconds": 5,
  "capture_seconds": 4,
  "rx_gain_db": 45,
  "tx_gain_db": -30,
  "tx_amplitude": 0.15,
  "tx_audio_tone_hz": 1000,
  "tx_fm_deviation_hz": 5000,
  "squelch_db": -120,
  "noise_gate_db": -120,
  "simulate": true
}
```

For a live cabled RF test, omit `simulate` and include:

```
{
  "confirm_live_tx": true
}
```

Response fields:

```
{
  "ok": true,
  "bounded": true,
  "loopback_demod": {
    "state": "complete",
    "rx_profile": "NOAA_NFM",
    "tx_profile": "TX_AUDIO_FM",
    "rx_frequency_hz": 915000000,
    "tx_frequency_hz": 915000000,
    "metrics": {
      "sample_count": 192000,
      "sample_rate_hz": 2400000,
      "duration_ms": 4000,
      "tx_mode": "fm",
      "carrier_offset_hz": 25000,
      "rx_rms_dbfs": -78.0,
      "rx_peak_dbfs": -63.0,
      "demod_sample_count": 192000,
      "demod_rms": 0.31,
      "tone_hz": 1000,
      "detected_tone_hz": 1000,
      "tone_dbfs": -58.0,
      "reference_dbfs": -66.0,
      "tone_snr_db": 8.0,
      "pass_snr_db_min": 6.0,
      "passed": true,
      "backend": "pluto-loopback-backend"
    }
  }
}
```

Use `GET /radio/loopback/demod/status` to retrieve the last diagnostic result.

`POST /radio/loopback/ft8`

Runs the bounded cabled FT8 decoder test. Connect TX1 to RX1 through a 30 dB attenuator. The standard 40 m FT8 dial frequency is 7.074 MHz; because it is below the Pluto tuning range, this diagnostic simulates a +125 MHz upconverter and tunes both RF ports to 132.074 MHz.

```
{
  "source_frequency_hz": 7074000,
  "frequency_translation_hz": 125000000,
  "attenuator_db": 30,
  "tx_ft8_text": "CQ K1ABC FN42",
  "confirm_live_tx": true
}
```

TX is bounded to 30 seconds and RX capture to 32 seconds so the UTC-synchronized transmitter and receiver see at least one complete 15-second FT8 slot and the decoder has time to publish its result. Live TX requires `confirm_live_tx=true`, and requests declaring less than 30 dB attenuation are rejected. `metrics.passed` is true only when the firmware decoder returns the exact transmitted message. Poll `GET /radio/loopback/ft8/status` for the last result. Never use a direct unattenuated TX-to-RX cable.

Observed cabled-loopback result with TX1 connected to RX1 through 30 dB:

```
{
  "state": "complete",
  "source_frequency_hz": 7074000,
  "frequency_translation_hz": 125000000,
  "rx_frequency_hz": 132074000,
  "tx_frequency_hz": 132074000,
  "attenuator_db": 30,
  "metrics": {
    "passed": true,
    "decoded_texts": ["CQ K1ABC FN42"],
    "ft8_decode": {
      "slot_utc": "2026-07-19T02:35:45Z",
      "slots_completed": 1,
      "slots_dropped": 0,
      "decode_time_ms": 112.992,
      "candidate_count": 15,
      "message_count": 1,
      "messages": [
        {
          "time_offset_s": 1.44,
          "audio_frequency_hz": 1500.0,
          "sync_score": 23,
          "text": "CQ K1ABC FN42"
        }
      ]
    }
  }
}
```

POST /radio/loopback/cw

Runs a bounded closed-loop CW diagnostic. Firmware starts the CW RX audio path and bounded TX_CW inside the single-process `pluto-loopback-backend` full-duplex path. It verifies recovered RF energy/keying on the cabled loopback path, decodes the received CW envelope into Morse symbols/text, and leaves the normal browser audio path untouched.

Default receive/transmit pairing:


```
{
  "rx_profile": "UHF_CW_LOOPBACK",
  "tx_profile": "TX_CW",
  "frequency_hz": 915000000,
  "duration_seconds": 10,
  "capture_seconds": 9,
  "rx_gain_db": 35,
  "tx_gain_db": -35,
  "tx_amplitude": 0.08,
  "tx_cw_text": "CQ TEST",
  "tx_cw_wpm": 12,
  "squellch_db": -120,
  "noise_gate_db": -120,
  "confirm_live_tx": true
}
```

Response fields:

```
{
  "ok": true,
  "bounded": true,
  "loopback_cw": {
    "state": "complete",
    "mode": "cw",
    "rx_profile": "UHF_CW_LOOPBACK",
    "tx_profile": "TX_CW",
    "rx_frequency_hz": 915000000,
    "tx_frequency_hz": 915000000,
    "tx_cw_text": "CQ TEST",
    "tx_cw_wpm": 12,
    "metrics": {
      "sample_rate_hz": 2400000,
      "tx_mode": "cw",
      "rx_rms_dbfs": -78.0,
      "rx_peak_dbfs": -63.0,
      "pass_criteria": {
        "rx_peak_dbfs_min": -85.0
      }
    },
    "passed": true,
    "keying": {
      "decode_supported": true,
      "requested_wpm": 12,
      "estimated_wpm": 12,
      "estimated_unit_ms": 100,
      "timing_source": "auto",
      "expected_keyed_percent": 41.18,
      "keyed_percent": 45.0,
      "keying_segments": 28,
      "decoded_runs": "M318ms/3u S98ms/1u ...",
      "decoded_symbols": "-.-. --- / - . . . . - ",
      "decoded_text": "CQ TEST",
      "expected_text": "CQ TEST",
      "matched_expected": true
    },
    "decoded_symbols": "-.-. --- / - . . . . - ",
    "decoded_text": "CQ TEST"
  }
}
```

The loopback decoder uses an adaptive envelope threshold, automatic timing estimation from received mark/space runs, unit-scale smoothing, and automatic mark/space polarity correction because the cabled Pluto TX/RX path can show LO leakage during keyed-off gaps. Applications should use `metrics.keying.decode_supported`, `metrics.keying.decoded_text`, and `metrics.keying.matched_expected` to decide whether a loopback decode was valid. For repeated beacon-style CW streams, `matched_expected` is true when the decoded text begins with the expected text followed by end-of-text or a word space; the capture may include the first character(s) of the next repeat. `requested_wpm` is the requested/generated speed for loopback; `estimated_wpm` and `estimated_unit_ms` are

measured from the received timing and are the values the decoder used. For arbitrary off-air CW, applications should treat the decoded text as a best-effort diagnostic until a broader real-world CW test matrix is completed.

Transmit

POST /radio/tx/start

Request:

```
{
  "profile": "TX_TEST_TONE",
  "duration_seconds": 10,
  "frequency_hz": 915000000,
  "tx_mode": "fm",
  "tx_gain_db": -30,
  "tx_audio_tone_hz": 1200,
  "tx_fm_deviation_hz": 5000,
  "tx_amplitude": 0.05,
  "simulate": true
}
```

For live RF, omit `simulate` and add:

```
{
  "confirm_live_tx": true
}
```

Supported `tx_mode` values:

```
tone
carrier
am
fm
cw
ft8
```

Stock TX profiles:

```
TX_TEST_TONE
TX_AUDIO_AM
TX_AUDIO_FM
TX_CW
TX_FT8_LOOPBACK
```

Behavior:

- 1 Requires a TX-enabled profile.
- 1 Requires explicit confirmation for live RF.
- 1 Bounds duration, amplitude, tone frequency, sample rate, TX gain, audio rate, modulation index/deviation, CW text/speed, and FT8 message text.
- 1 Applies TX LO, TX bandwidth, TX sample rate, TX gain, and ENSM mode.
- 1 Runs `/usr/sbin/pluto-tx-backend`, currently a symlink to the small libiio backend also used by loopback.
- 1 Returns bounded IQ metrics plus TX audio source/rate/tone, file-read evidence, measured audio tone, and stores state in `/var/run/pluto-radio/tx.json`.

AM test-tone request:

```
{
  "profile": "TX_AUDIO_AM",
  "duration_seconds": 5,
  "tx_audio_source": "tone",
  "tx_audio_tone_hz": 1000,
  "tx_am_modulation_index": 0.8,
  "simulate": true
}
```

FM PCM-file request:

```
{
  "profile": "TX_AUDIO_FM",
  "duration_seconds": 5,
  "tx_audio_source": "file",
  "tx_audio_path": "/mnt/jffs2/tx-audio/test.pcm",
  "tx_audio_rate_hz": 8000,
  "tx_fm_deviation_hz": 5000,
  "simulate": true
}
```

CW request:

```
{
  "profile": "TX_CW",
  "duration_seconds": 5,
  "tx_cw_text": "CQ PLUTO",
  "tx_cw_wpm": 12,
  "simulate": true
}
```

POST /radio/tx/stop

Forces ENSM to a lert when possible and marks TX stopped.

Doppler

POST /radio/doppler/plan

Request:

```
{
  "profile": "SAT_AUDIO_NFM",
  "base_frequency_hz": 145800000,
  "start_utc": "2026-07-08T20:15:00Z",
  "stop_utc": "2026-07-08T20:27:00Z",
  "retune_interval_ms": 1000,
  "doppler_table": [
    { "offset_ms": 0, "frequency_hz": 145803200 },
    { "offset_ms": 1000, "frequency_hz": 145803050 }
  ]
}
```

Behavior:

- 1 The application owns TLEs, pass prediction, satellite selection, and table generation.
- 1 Firmware validates and stores the table.
- 1 Firmware applies the base profile/frequency and reports current/next retune state.
- 1 PLUTO_DOPPLER_MAX_POINTS and PLUTO_DOPPLER_MIN_INTERVAL_MS bound table size and retune cadence.

POST /radio/doppler/start

Starts /usr/sbin/pluto-doppler-worker.

POST /radio/doppler/tick

Runs one scheduler tick. This is useful for tests or an external supervisor.

POST /radio/doppler/stop

Stops the worker and leaves the last plan stored.

CLI Equivalents

The HTTP API and CLI share the same validation code:

```
pluto-radio-api status
pluto-radio-api health
pluto-radio-api profiles
pluto-radio-api apply PROFILE [FREQUENCY_HZ]
pluto-radio-api tune FREQUENCY_HZ
pluto-radio-api stop
pluto-radio-api audio-start PROFILE [--simulate] [key=value...]
pluto-radio-api audio-stop
pluto-radio-api audio-status
pluto-radio-api capture-start PROFILE [--simulate]
pluto-radio-api capture-list
pluto-radio-api spectrum-status
pluto-radio-api spectrum-snapshot PROFILE [--simulate]
pluto-radio-api spectrum-top PROFILE [--simulate]
pluto-radio-api loopback-status
pluto-radio-api loopback-start LOOPBACK_TEST [--simulate] [--confirm-live-tx] [key=value...]
pluto-radio-api tx-status
pluto-radio-api tx-start TX_TEST_TONE [--simulate] [--confirm-live-tx] [key=value...]
pluto-radio-api tx-stop
pluto-radio-api doppler-status
pluto-radio-api doppler-plan [PROFILE]
pluto-radio-api doppler-start
pluto-radio-api doppler-stop
pluto-radio-api doppler-tick
```

Onboard Test Page

The firmware includes an app-builder test page:

```
http://192.168.2.1/api-test.html
```

It exercises logical API routes from the browser and includes safe simulated tests for health, profiles, audio, spectrum, loopback, and TX. The page sends requests through the same-origin lighttpd proxy and shows the matching on-device direct URL using `http://127.0.0.1:8081`. Live TX remains guarded by an explicit checkbox and should only be used in a controlled RF setup.

Validation

Host-side validation:

```
bash scripts/validate-pluto-radio-api.sh
bash scripts/check-pluto-build-hygiene.sh
python3 scripts/check-firmware-size-budget.py
```

The validation script:

- 1 Compiles Python helpers.
- 1 Loads all profile JSON files.

- 1 Exercises status, health, apply, tune, simulated audio, spectrum, loopback, TX, Doppler, and invalid-input paths.
- 1 Checks that the DSP reference backend can produce PCM from synthetic IQ.

The final acceptance check for release remains a full Buildroot image build and target-rootfs inspection, because the production backends link against target libraries.